Android Terminal Connection Guide

The **Android Terminal Connection Guide** provides a step-by-step overview for integrating and connecting an INGENICO MOBY 5500 Bluetooth terminal with your Android point-of-sale (POS) application. This guide will cover the following areas:

- **Prepare Merchant & Terminal Information**: Define the MerchantInformation and DeviceInformation objects. These objects are required to initialize the TerminalManager object.
- **Create and Initialize the Terminal Manager Object**: Creating and configuring the terminal manager with the necessary context.
- **Bluetooth Discovery and Pairing**: Discovering nearby terminals, selecting a device, and completing LED-based pairing.
- **Response Handling**: Identifying and resolving common connection issues such as permission errors or pairing failures.

Prerequisites

Before you begin integrating the Deluxe POS SDKs, ensure you have the following prerequisites ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - Android Merchant Authorization & Accounts Guide

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Setup Requirement

ON THIS PAGE

Prerequisites

Setup Requirement

Prepare Merchant, Terminal Information and Pos App Details

Create and Initialize the Terminal Manager Object

Bluetooth Discovery and Pairing

Response Handling

throughout the connection process.

Connecting to the Ingenico Moby5500 terminal requires the following Android application permissions in the manifest file of the mobile application and are mandatory for Terminal Connection.

```
<?xml version="1.0" encoding="utf-8"
<manifest
xmlns:android="http://schemas.android.com/apk/res/andr
oid"
xmlns:tools="http://schemas.android.com/tools">
<uses-permission
android:name="android.permission.INTERNET"/>
<uses-permission
android:name="android.permission.BLUETOOTH"/>
<uses-permission
android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"
/>
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATIO
N"/>
```

After integrating all the AAR files into your project, you will be able to access all the classes provided by the POSSDK. To use any class from the POSSDK, you need to import it into your own class.

Prepare Merchant, Terminal Information and Pos App Details

Define the MerchantInformation, DeviceInformation and PosAppDetails objects. These objects are required to initialize the TerminalManager object.

Merchant Information:

```
val merchantInfo = MerchantInformation(
companyId: "000005",// This should be 000005
mid: "",// This should be mid of your merchant account
tid: "",//This Should be tid of your mechant account
bankId: "",//This can be left empty as of now.
merchantDBA: "",//This should be dba of your merchant
account.
accountId: "",//This should be accessToken of the
merchant account.
routing : "",
bearerToken: "" // This should be  bearer token
received from the MerchatAuthorizationService
)
```

Terminal Information:

The SDK currently only supports the Ingenico Moby5500 terminal with a Bluetooth connection.

```
//This is the INGENICO MOBY 5500 bluetooth connection
terminalinformation
val terminalInfo = TerminalInformation(
  manufacturer = TerminalManufacturer.INGENICO,
// Manufacturer of the terminal (e.g., INGENICO)
  model = TerminalModel.MOBY5500,
// Terminal model (e.g., MOBY5500)
  connectionType = TerminalConnectionType.BLUETOOTH,
// Connection type used (e.g., BLUETOOTH)
  connectionStatus = TerminalConnectionStatus.UNKNOWN,
// Current connection status (e.g., CONNECTED,
DISCONNECTED, UNKNOWN)
  terminalAppName = "DlxCrtAnd",
// Please leave this value for now
  terminalAppVersion = "0001",
// Please leave this value for now
  terminalName = "",
```

Pos App Details:

Define the PosAppDetails with App Name and App version.

```
var posAppDetails = PosAppDetails(
posAppName = "DLX_SDK_1",
posAppVersion = "010101"
)
```

If a valid terminalName is provided, the TerminalManager will attempt to connect directly to that specific terminal. Otherwise, it will initiate a Bluetooth discovery phase to detect all nearby Moby devices and return a list of available terminals.

Create and Initialize the Terminal Manager Object

Step 1: Create TerminalManager Object

```
var terminalManager =
TerminalManager(context,terminalInformation,
merchantInformation,posAppDetails)
```

Step 2: Implement Terminal Connection Callback

```
terminalManager.initializeTerminalManager(
applicationContext,terminalConnectionCallback
)
```

```
var terminalConnectionCallback = object :
TerminalConnectionCallback {

   override fun
onBluetoothDiscoveredDevices(availableReaderList:
ArrayList<String>) {
       // Handle the list of available Bluetooth
devices discovered during scanning
   }

   override fun onBluetoothDiscoveryFailed(error:
String) {
       // Handle failure in discovering Bluetooth
devices
   }

   override fun onConnectionFailed(
```

Bluetooth Discovery and Pairing

Step 1: Call connect() to start the bluetooth connection process.

The process begins by scanning for nearby Bluetooth devices.

```
terminalManager.connect(context)
```

If the terminalName in the TerminalInformation object is left empty, the connect() method will initiate a Bluetooth discovery process to locate all nearby Moby5500 devices. It will then trigger the onBluetoothDiscoveryCompleted(devices: List<String>) callback, which returns a list of available Moby5500 terminal names detected in the vicinity.

If a specific terminalName is provided in the TerminalInformation object, the connect() method will still initiate Bluetooth discovery but will automatically select the specified terminal and proceed with pairing—bypassing the need to call connect(selectedDeviceName) separately.

Step 2: Discover and Select a Terminal for Connection

When the onBluetoothDiscoveryCompleted(devices: List<String>) method is triggered, it provides a list of nearby moby5500 devices. Implement this method to display the devices in your application and

```
override fun
onBluetoothDiscoveredDevices(availableReaderList:
ArrayList<String>) {
this.devices = devices
// Handle the list of available Bluetooth devices
discovered during scanning
// Save and display the list of discovered devices
// Show a UI to let the user select a device
}
```

Once the user selects a device from the displayed list, pass the selected device name to the connectBluetoothDevice method to initiate the LED pairing process.

```
val selectedDeviceName = "Device_Name" // Replace with
the user-selected device
terminalManager.connect(selectedDeviceName)
```

Step 3: Implement LED Pairing

After invoking the connectBluetoothDevice method, the TerminalManager begins the pairing process. For Moby5500 terminals, LED pairing is initiated, and the onDeviceLedConnectionUpdates(ledSequence: List<TerminalLedStatus>) method is called with LED status updates.

Display LED Statuses:

Use the ledSequence to show the status of the terminal's LEDs (red, yellow, orange, blue) on your app's UI.

User Actions:

Provide the following options for the merchant:

- **Confirm Pairing**: If the LED indications match between the app and terminal.
- **Repeat Pairing**: To restart the LED pairing process.
- **Cancel Pairing**: To abort the pairing and terminate the connection request.

```
override fun onDeviceLedConnectionUpdates(ledSequence:
List<LedStatus>) {
// Display LED pairing statuses and provide user
actions (Confirm, Retry, Cancel).
// In the UI, based on the ledSequence, show the
appropriate ui and display three buttons: Confirm,
Retry, and Cancel.
// When the user selects an action, call the
```

```
// User actions:
terminalManager.confirmBluetoothLedPairing()  //
Confirm pairing
```

Response Handling

On Success: When pairing succeeds, the terminal moves to the connected state, and the onConnectionSucceeded() method is called.

On Failure: If the connection fails, the onConnectionFailed() method is triggered with the reason for the failure.

```
override fun onConnectionSucceeded() {
// Terminal successfully connected and ready for
transactions
}

override fun onConnectionFailed(connectionStatus:
TerminalConnectionStatus, errorMessage: String) {
// Handle connection failure or cancellation
}
```

connectionStatus indicates the last available status of the terminal, helping to identify what exact reason the connection is failed.

Below are some of the possible terminal connection failure statuses:

TerminalConnectionStatus.BLUETOOTH_DISCOVERY_FAILED: When no bluetooth devices found or when bluetooth permission is not enabled.

TerminalConnectionStatus.BLUETOOTH_DEVICES_NOT_FOUND: This status occurs when no Bluetooth devices are found or when Bluetooth permissions are not enabled. It typically appears when a connection attempt is made during a transaction workflow.

TerminalConnectionStatus.PAIRING_FAILED: This status indicates that pairing failed due to unknown hardware issues. While it's an uncommon error, it has been observed on some Moby5500 devices.

TerminalConnectionStatus.BLUETOOTH_PERMISSIONS_ERROR: This status indicates a pairing failure due to a terminal bluetooth issue. If this error occurs, the user must manually go to the Android Bluetooth settings, enable it, and then try connecting again.

deluxe. connect

Payments API

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave,
Minneapolis, MN 55402.

© 2026 Deluxe Corporation